

PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN LANJUT
SEMESTER GENAP T.A. 2025/2026
LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NAMA/NIM : Rifani Juniarti/ 123250117
Tessa Hasiantha / 123250135
KELAS/PLUG : IF - F
NAMA ASISTEN : Bintang Shada Kawibya Putra (123240247)
Muhammad Daud Akyaz (123240081)

PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026

HALAMAN PENGESAHAN
LAPORAN PROYEK AKHIR

Disusun oleh :

Rifani Juniarti 123250117
Tessa Hasiantha 123250135

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Algoritma dan Pemrograman
Pada Tanggal :

Asisten Praktikum

Bintang Shada Kawibya Putra
NIM. 123240247

Asisten Praktikum

Muhammad Daud Akyaz
NIM. 123240081

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya kami dapat menyelesaikan Laporan Akhir Praktikum Algoritma dan Pemrograman Lanjut ini dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan evaluasi dari kegiatan praktikum yang telah kami laksanakan selama satu semester. Melalui praktikum ini, kami memperoleh pengalaman dalam menerapkan konsep algoritma dan pemrograman untuk menyelesaikan berbagai permasalahan serta mengembangkan proyek yang menjadi bagian dari tugas akhir praktikum.

Kami mengucapkan terima kasih para asisten praktikum yang telah memberikan bimbingan, arahan, dan bantuan selama proses praktikum hingga penyusunan laporan ini. Ucapan terima kasih juga kami sampaikan kepada seluruh anggota kelompok yang telah bekerja sama dan berkontribusi dalam menyelesaikan tugas serta laporan akhir ini.

Kami menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kami mengharapkan kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan ini di masa mendatang. Semoga laporan akhir ini dapat memberikan manfaat dan menambah wawasan bagi para pembaca serta menjadi referensi yang berguna dalam mempelajari Algoritma dan Pemrograman Lanjut.

Yogyakarta, 1 Juni 2026

Penyusun I



Rifani Juniarti

NIM. 123250117

Penyusun II



Tessa Hasiantha

NIM. 123250135

DAFTAR ISI

HALAMAN PENGESAHAN	ii
KATA PENGANTAR	iii
DAFTAR ISI	iv
BAB I	1
BAB II.....	3
BAB III	4
3.1. Implementasi Materi.....	4
3.2. Penjelasan Fitur	8
3.3. <i>Screenshot</i> Program.....	16
3.4. <i>Source Code</i> Lengkap.....	26
BAB IV	38
4.1. Jadwal Pengerjaan	38
4.2. Pembagian Tugas.....	38
BAB V	40
5.1 Kesimpulan.....	40
5.2 Saran	40
DAFTAR PUSTAKA	41

BAB I

Hotel Management System

Program yang dibuat merupakan *Hotel Management System* berbasis bahasa pemrograman C++ yang berfungsi untuk membantu pengelolaan pemesanan kamar hotel secara sederhana. Program ini dirancang untuk mencatat data tamu, mengelola status kamar, melakukan proses *check-in* dan *check-out*, serta menghitung pendapatan hotel. Hotel dalam program ini memiliki 3 lantai dengan 5 kamar pada setiap lantainya, sehingga tersedia total 15 kamar yang dapat dipesan oleh tamu. Setiap tamu yang melakukan pemesanan akan memperoleh ID secara otomatis, kemudian data tamu tersebut disimpan ke dalam file sehingga informasi tetap tersimpan meskipun program ditutup dan dijalankan kembali.

Program ini memiliki beberapa fitur utama yang mendukung proses manajemen hotel. Fitur pertama adalah Lihat Status Kamar, yang digunakan untuk menampilkan seluruh kamar beserta statusnya, apakah masih kosong atau sudah terisi. Dengan fitur ini, petugas hotel dapat dengan mudah mengetahui kamar yang tersedia untuk dipesan. Fitur berikutnya adalah *Booking* Kamar, yang memungkinkan petugas memasukkan data tamu seperti nama, nomor kamar, dan lama menginap. Sistem akan memeriksa ketersediaan kamar yang dipilih dan menyimpan data tamu apabila kamar masih kosong. Setelah proses berhasil, status kamar akan berubah menjadi terisi.

Selain itu, terdapat fitur *Check Out* Kamar yang digunakan ketika tamu selesai menginap. Pada proses ini, petugas hanya perlu memasukkan ID tamu, kemudian sistem akan menampilkan informasi tamu beserta total biaya yang harus dibayar berdasarkan lama menginap. Setelah transaksi selesai, status tamu akan berubah menjadi *check-out* dan kamar yang sebelumnya ditempati akan kembali tersedia untuk digunakan oleh tamu lain. Program juga menyediakan fitur Cari Data Tamu yang memungkinkan pencarian berdasarkan ID, nama, nomor kamar, maupun menampilkan data tamu yang terakhir melakukan pemesanan. Fitur ini sangat membantu dalam menemukan informasi tamu secara cepat dan efisien.

Untuk kebutuhan administrasi, program dilengkapi dengan fitur Laporan Data Tamu yang dapat menampilkan daftar tamu aktif, tamu yang sudah *check-out*, maupun seluruh data tamu yang tersimpan. Selain itu, terdapat fitur Hitung Total Pendapatan yang digunakan untuk menghitung total pemasukan hotel dari seluruh transaksi yang terjadi. Sistem akan menampilkan rincian biaya setiap tamu, jumlah keseluruhan pendapatan, serta rata-rata

pendapatan yang diperoleh per tamu. Dalam program ini, biaya kamar ditetapkan sebesar Rp300.000 per malam.

Dalam pembangunannya, program menerapkan berbagai konsep dasar pemrograman yang dipelajari pada mata kuliah algoritma dan struktur data. Data tamu disimpan menggunakan struktur (*struct*) sehingga informasi seperti ID, nama, kamar, lama menginap, dan status dapat dikelola dengan lebih terorganisir. Program juga menggunakan *array* untuk menyimpan data tamu dan status kamar hotel. Untuk mempermudah pengelolaan program, setiap proses dibagi ke dalam beberapa fungsi (*function*) yang memiliki tugas masing-masing, seperti fungsi *booking*, *check-out*, pencarian, dan laporan.

Pada bagian pencarian data, program menerapkan dua metode yaitu *Binary Search* untuk pencarian berdasarkan ID tamu dan *Linear Search* untuk pencarian berdasarkan nama maupun nomor kamar. Sementara itu, proses pengurutan data menggunakan algoritma *Bubble Sort* agar data tamu tersusun berdasarkan ID. Program juga memanfaatkan konsep rekursi pada fungsi perhitungan biaya menginap, di mana fungsi memanggil dirinya sendiri hingga mencapai kondisi dasar tertentu. Selain itu, digunakan pula konsep file handling melalui *ifstream* dan *ofstream* untuk membaca, menyimpan, dan memperbarui data tamu dalam file `data_tamu.txt`.

Secara keseluruhan, program *Hotel Management System* ini mampu membantu proses pengelolaan hotel secara lebih efektif dan terstruktur. Dengan adanya fitur pemesanan kamar, pencarian data, laporan tamu, perhitungan pendapatan, serta penyimpanan data permanen, program ini dapat menjadi solusi sederhana untuk mengurangi pencatatan manual dan meningkatkan efisiensi dalam pengelolaan operasional hotel.

BAB II

TUJUAN PROYEK AKHIR

Tujuan dari proyek akhir ini adalah untuk menerapkan serta mengembangkan kemampuan pemrograman yang telah dipelajari selama perkuliahan, khususnya dalam penggunaan bahasa C++, algoritma, dan struktur data untuk menyelesaikan permasalahan nyata. Melalui proyek ini, mahasiswa dapat memahami penerapan berbagai konsep pemrograman seperti *struct*, *array*, fungsi, *searching*, *sorting*, rekursi, dan file handling dalam sebuah aplikasi yang terintegrasi. Selain sebagai sarana pembelajaran, proyek ini juga bertujuan untuk melatih kemampuan analisis, perancangan sistem, serta penyelesaian masalah secara logis dan sistematis. Oleh karena itu, dibuatlah aplikasi *Hotel Management System* yang berfungsi untuk membantu pengelolaan pemesanan kamar hotel secara lebih efektif dan terorganisir. Aplikasi ini dirancang untuk mempermudah proses pencatatan data tamu, pengecekan ketersediaan kamar, pemesanan kamar, proses *check-out*, pencarian data tamu, pembuatan laporan, hingga perhitungan pendapatan hotel secara otomatis. Dengan adanya aplikasi ini, proses pengelolaan hotel yang sebelumnya dilakukan secara manual dapat menjadi lebih cepat, akurat, dan efisien, serta mengurangi risiko kesalahan dalam pencatatan data karena seluruh informasi tersimpan secara digital dan dapat digunakan kembali melalui sistem penyimpanan file.

BAB III

PEMBAHASAN

3.1. Implementasi Materi

3.1.1 *Struct*

Materi *struct* diimplementasikan untuk membuat tipe data dengan bentuk baru bernama *Tamu*. *Struct* digunakan untuk mengelompokkan data informasi setiap tamu hotel (seperti ID, nama, posisi lantai, nomor kamar, lama menginap, dan status aktif) ke dalam satu kesatuan entitas agar pengelolaan data menjadi lebih terstruktur.

```
struct Tamu {  
    int id;  
    string nama;  
    int lantai;  
    int kamar;  
    int lamaInap;  
    bool status;  
};
```

3.1.2 *Array*

Array digunakan untuk menyimpan kumpulan data. Pada proyek ini, digunakan *array* satu dimensi bertipe *struct* *Tamu* untuk menampung seluruh data reservasi, serta *array* dua dimensi `hotel[3][5]` bertipe *boolean* untuk menggambarkan denah dan status keterisian kamar (3 lantai dan 5 kamar per lantai).

```
bool hotel[3][5] = {0};  
const int maxTamu = 200;  
Tamu tamu[maxTamu];
```

3.1.3 *Searching*

Fitur *searching* pada program ini mengimplementasikan dua metode, yaitu *binary search* dan *linear search*. *Binary search* digunakan untuk mencari data tamu berdasarkan ID. Metode ini membagi data menjadi dua bagian secara berulang, sehingga proses pencarian menjadi lebih cepat, dengan syarat data harus terurut terlebih dahulu. Sedangkan *linear search* digunakan untuk mencari data berdasarkan nama tamu dan nomor kamar dengan cara mencocokkan baris data satu per satu dari awal sampai akhir.


```

if (cari == 1) { // binary search
    int keyword;
    cout << "Masukkan ID : ";
    cin >> keyword;

    bubbleSort();

    int awal = 0;
    int akhir = jumlahTamu - 1;
    int tengah;

    while (awal <= akhir) {
        tengah = (awal + akhir) / 2;

        if (tamu[tengah].id == keyword) {
            printData(&tamu[tengah]);
            ditemukan = true;
            break;
        } else if (tamu[tengah].id < keyword) {
            awal = tengah + 1;
        } else {
            akhir = tengah - 1;
        }
    }
} else if (cari == 2) { // linear search
    string keyword;
    cout << "Masukkan Nama : ";
    cin.ignore();
    getline(cin, keyword);
    string upperKeyword = stringToupper(keyword);
    for (int i = 0; i < jumlahTamu; i++) {
        if (stringToupper(tamu[i].nama) == upperKeyword) {
            printData(&tamu[i]);
            ditemukan = true;
        }
    }
} else if (cari == 3) { // linear search
    string keyword;
    cout << "Masukkan Nomor Kamar (Contoh: 1A) : ";
    cin >> keyword;
    if (keyword.length() == 2) {
        int lantaiCari = keyword[0] - '1';
        int kamarCari = getIndexKamar(keyword[1]);
        for (int i = 0; i < jumlahTamu; i++) {
            if (tamu[i].lantai == lantaiCari && tamu[i].kamar
== kamarCari) {
                printData(&tamu[i]);
                ditemukan = true;
            }
        }
    } else {
        cout << "Format kamar salah!\n";
    }
} else if (cari == 4) {

```

```

        printData(&tamu[jumlahTamu - 1]);
        ditemukan = true;
    } else {
        cout << "Pilihan tidak valid!\n";
    }
}

```

3.1.4 Sorting

Materi sorting diimplementasikan dengan metode *bubble sort* untuk mengurutkan data tamu berdasarkan ID secara urutan naik (*ascending*). Pengurutan ini dilakukan sebelum menampilkan laporan data tamu dan sebelum menjalankan fungsi *Binary Search* agar algoritma pencarian biner dapat berfungsi dengan valid.

```

void bubbleSort() {
    for (int i = 0; i < jumlahTamu - 1; i++) {
        for (int j = 0; j < jumlahTamu - i - 1; j++) {
            if (tamu[j].id > tamu[j + 1].id) {
                Tamu temp = tamu[j];
                tamu[j] = tamu[j + 1];
                tamu[j + 1] = temp;
            }
        }
    }
}

```

3.1.5 Rekursif

Fungsi rekursif diterapkan pada fungsi *hitungBiaya()* untuk menghitung total biaya penginapan secara berulang berdasarkan jumlah malam menginap. Fungsi akan terus memanggil dirinya sendiri dengan mengurangi jumlah hari (*hari - 1*) hingga mencapai *base case* atau kondisi berhenti, yaitu ketika variabel *hari* bernilai kurang dari atau sama dengan nol.

```

int hitungBiaya(int hari) {
    int harga = 300000;
    if(hari <= 0)
        return 0;

    return harga + hitungBiaya(hari - 1);
}

```

3.1.6 Pointer

Pada proyek ini, pointer diimplementasikan sebagai parameter pada fungsi *printData(Tamu* t)* dan *saveFile(Tamu* t)*. Penggunaan pointer ini bertujuan untuk menerapkan mekanisme *pass-by-reference*, sehingga fungsi dapat

mengakses data objek Tamu langsung dari alamat memorinya tanpa perlu melakukan duplikasi data, sehingga menghemat penggunaan memori.

```
void printData(Tamu* t) {
    char noKamar = getAbjadKamar(t->kamar);
    string status = (t->status) ? "Aktif/Menginap" : "Sudah Check Out";

    cout << "-----\n"
        << "ID          : " << t->id << "\n"
        << "Nama         : " << t->nama << "\n"
        << "Lantai       : " << t->lantai + 1 << "\n"
        << "Kamar        : " << noKamar << "\n"
        << "Lama Inap    : " << t->lamaInap << "\n"
        << "Status       : " << status << "\n";
}
```

3.1.7 Operasi File

Operasi file menggunakan file I/O (*ifstream* dan *ofstream*) untuk menyimpan data tamu ke dalam file eksternal bernama data_tamu.txt. Fitur ini memastikan data transaksi hotel tidak hilang saat aplikasi ditutup (*data persistence*). Fungsi loadFile() digunakan untuk membaca data saat program dijalankan, sedangkan saveFile() dan rewriteFile() digunakan untuk menulis data baru atau memperbarui status kamar tamu.

```
void loadFile() {
    ifstream file("data_tamu.txt");
    if (file.is_open()){
        cout << "File ditemukan. Memuat data...\n";
        int maxId = 0;
        while (jumlahTamu < maxTamu && file >>
            tamu[jumlahTamu].id >> tamu[jumlahTamu].nama >>
            tamu[jumlahTamu].lantai >> tamu[jumlahTamu].kamar >>
            tamu[jumlahTamu].lamaInap >> tamu[jumlahTamu].status) {

            tamu[jumlahTamu].nama =
            underscoreToSpasi(tamu[jumlahTamu].nama);

            if (tamu[jumlahTamu].status) {
                hotel[tamu[jumlahTamu].lantai][tamu[jumlahTamu].kamar] = 1;
            }

            if (tamu[jumlahTamu].id > maxId) {
                maxId = tamu[jumlahTamu].id;
            }

            jumlahTamu++;
        }
    }
}
```

```

    }
    nextId = maxId + 1;
    file.close();
} else {
    cout << "File tidak ditemukan! Memulai program dengan
data kosong\n";
}
system("pause");
}

bool saveFile(Tamu* t) {
    ofstream file("data_tamu.txt", ios::app);

    if(!file.is_open()){
        return false;
    } else {
        file << t->id << " "
            << spasiToUnderscore(t->nama) << " "
            << t->lantai << " "
            << t->kamar << " "
            << t->lamaInap << " "
            << t->status
            << endl;
        file.close();
        return true;
    }
}

void rewriteFile() {
    ofstream file("data_tamu.txt", ios::trunc);

    if(file.is_open()){
        for (int i = 0; i < jumlahTamu; i++) {
            file << tamu[i].id << " "
                << spasiToUnderscore(tamu[i].nama) << " "
                << tamu[i].lantai << " "
                << tamu[i].kamar << " "
                << tamu[i].lamaInap << " "
                << tamu[i].status
                << endl;
        }
        file.close();
    }
}
}

```

3.2. Penjelasan Fitur

3.2.1 Fitur Lihat Status Kamar

Fitur ini diimplementasikan menggunakan *array* dua dimensi `hotel[3][5]` yang merepresentasikan kondisi seluruh kamar hotel. Program melakukan perulangan pada setiap lantai dan kamar untuk menampilkan status kamar, yaitu KOSONG

jika belum ditempati dan TERISI jika sedang digunakan oleh tamu. Dengan fitur ini, pengguna dapat melihat ketersediaan kamar secara langsung sebelum melakukan pemesanan.

```
void lihatKamar(){
    header();
    cout << "\n----- Lihat Kamar -----"
    --\n\n";

    string statusKamar;
    for (int i = 0; i < 3; i++){
        for (int j = 0; j < 5; j++){
            statusKamar = hotel[i][j] ? "TERISI" : "KOSONG";
            cout << "[Kamar " << i+1 << getAbjadKamar(j) << " "
            << statusKamar << "]\n";
        }
        cout << endl;
    }

    cout << endl;
    system("pause");
}
```

3.2.2 Fitur *Booking* Kamar

Fitur *booking* kamar diimplementasikan dengan menerima input data tamu berupa nama, nomor kamar, dan lama menginap. Sistem melakukan validasi terhadap format nomor kamar dan memeriksa apakah kamar yang dipilih masih tersedia. Jika valid, data tamu disimpan ke dalam struktur Tamu, status kamar diubah menjadi terisi, dan data disimpan ke file data_tamu.txt agar dapat digunakan kembali saat program dijalankan.

```
void bookingKamar(){
    header();
    cout << "\n----- Booking Kamar -----"
    ---\n\n";

    if (jumlahTamu >= maxTamu) {
        cout << "Maaf, kapasitas tamu penuh (maksimal " << maxTamu
        << " tamu)!\n";
        system("pause");
        return;
    }

    Tamu inputTamu;
    string noKamar;

    cout << "Data tamu :\n";
```

```

        cout << "-----\n";

        inputTamu.id = nextId;
        cout << "ID : " << inputTamu.id << " (Otomatis)\n";

        cout << "Nama : ";
        cin.ignore();
        getline(cin, inputTamu.nama);

        bool kamarValid = false;
        do {
            cout << "Nomor Kamar (Contoh: 1A): ";
            cin >> noKamar;

            if (noKamar.length() != 2) {
                cout << "Format salah harus 2 karakter (Contoh: 1A).\n";
                continue;
            }

            inputTamu.lantai = noKamar[0] - '1';
            inputTamu.kamar = getIndexKamar(noKamar[1]);

            if (inputTamu.lantai < 0 || inputTamu.lantai > 2 ||
                inputTamu.kamar == -1 || toupper(noKamar[1]) > 'E') {
                cout << "Kamar tidak ada! Pilih lantai 1-3 dan abjad A-E.\n";
                continue;
            }

            if (hotel[inputTamu.lantai][inputTamu.kamar] == true) {
                cout << "Kamar " << noKamar << " sudah terisi! Silakan pilih yang kosong.\n";
                continue;
            }

            kamarValid = true;
        }while (kamarValid == false);

        cout << "Lama Inap : "; cin >> inputTamu.lamaInap;
        inputTamu.status = true;

        if (saveFile(&inputTamu) == true) {
            tamu[jumlahTamu] = inputTamu;
            hotel[inputTamu.lantai][inputTamu.kamar] = 1;
            jumlahTamu++;
            nextId++;
            cout << "Berhasil booking kamar\n";
        } else {
            cout << "Gagal booking kamar\n";
        }
    }

```

```

        cout << "-----\n\n";
        system("pause");
    }

```

3.2.3 Fitur *Check Out* Kamar

Fitur ini diimplementasikan dengan mencari data tamu berdasarkan ID yang dimasukkan pengguna. Setelah data ditemukan, sistem menghitung biaya menginap berdasarkan lama menginap menggunakan fungsi rekursif, kemudian menampilkan *invoice* pembayaran. Setelah proses selesai, status tamu diubah menjadi sudah *check-out* dan kamar yang ditempati dikosongkan kembali sehingga dapat digunakan oleh tamu lain.

```

void checkOutKamar() {
    header();
    cout << "\n----- Check Out Kamar -----
---\n\n";

    if (jumlahTamu == 0) {
        cout << "Data tamu kosong. Tidak ada yang bisa
dichcheckout.\n";
        system("pause");
        return;
    }

    int id;
    cout << "Masukkan ID Tamu : ";
    cin >> id;

    bool ditemukan = false;
    for (int i = 0; i < jumlahTamu; i++) {
        if (tamu[i].id == id) {
            ditemukan = true;
            if (!tamu[i].status) {
                cout << "Tamu dengan ID " << id << " sudah check
out.\n";
            } else {
                int tagihan = hitungBiaya(tamu[i].lamaInap);

                cout << "\n=====
\n";
                cout << "                               INVOICE CHECK OUT
\n";
                cout << "=====
\n";
                cout << "ID          : " << tamu[i].id << "\n";
                cout << "Nama          : " << tamu[i].nama << "\n";
                cout << "Kamar          : " << tamu[i].lantai + 1
<< getAbjadKamar(tamu[i].kamar) << "\n";

```

```

        cout << "Lama Inap   : " << tamu[i].lamaInap <<
" malam\n";
        cout << "Total Biaya : Rp " << tagihan << "\n";
        cout << "===== \n";

        tamu[i].status = false;
        hotel[tamu[i].lantai][tamu[i].kamar] = 0;
        rewriteFile();

        cout << "\nCheck Out Berhasil! Kamar sudah
dikosongkan.\n";
    }
    break;
}

if (!ditemukan) {
    cout << "Tamu dengan ID " << id << " tidak ditemukan.\n";
}

cout << "-----
\n\n";
system("pause");
}

```

3.3.4 Fitur Cari Data Tamu

Fitur pencarian data tamu memungkinkan pengguna mencari informasi berdasarkan beberapa kategori. Untuk pencarian berdasarkan ID digunakan algoritma *Binary Search*, sedangkan pencarian berdasarkan nama dan nomor kamar menggunakan *Linear Search*. Selain itu, tersedia pilihan untuk menampilkan data tamu yang terakhir melakukan *booking*. Hasil pencarian akan menampilkan informasi lengkap tamu yang ditemukan.

```

void cariTamu(){
    header();
    cout << "\n----- Cari Data Tamu -----
-----\n\n";

    if (jumlahTamu == 0) {
        cout << "Data tamu kosong. Tidak ada data yang bisa
dicari. Silakan booking kamar\n";
        system("pause");
        return;
    }

    int cari;
    cout << "Pilih kategori pencarian:\n"
        << "1. ID\n"
        << "2. Nama\n"

```



```

        << "3. Kamar\n"
        << "4. Terakhir Booking\n"
        << "Pilih menu [1-4] : ";
    cin >> cari;

    bool ditemukan = false;
    cout << "\nHasil Pencarian:\n";

    if (cari == 1) { // binary search
        int keyword;
        cout << "Masukkan ID : ";
        cin >> keyword;

        bubbleSort();

        int awal = 0;
        int akhir = jumlahTamu - 1;
        int tengah;

        while (awal <= akhir) {
            tengah = (awal + akhir) / 2;

            if (tamu[tengah].id == keyword) {
                printData(&tamu[tengah]);
                ditemukan = true;
                break;
            } else if (tamu[tengah].id < keyword) {
                awal = tengah + 1;
            } else {
                akhir = tengah - 1;
            }
        }
    } else if (cari == 2) { // linear search
        string keyword;
        cout << "Masukkan Nama : ";
        cin.ignore();
        getline(cin, keyword);
        string upperKeyword = stringToupper(keyword);
        for (int i = 0; i < jumlahTamu; i++) {
            if (stringToupper(tamu[i].nama) == upperKeyword) {
                printData(&tamu[i]);
                ditemukan = true;
            }
        }
    } else if (cari == 3) { // linear search
        string keyword;
        cout << "Masukkan Nomor Kamar (Contoh: 1A) : ";
        cin >> keyword;
        if (keyword.length() == 2) {
            int lantaiCari = keyword[0] - '1';
            int kamarCari = getIndexKamar(keyword[1]);
            for (int i = 0; i < jumlahTamu; i++) {
                if (tamu[i].lantai == lantaiCari &&
                    tamu[i].kamar == kamarCari) {

```

```

        printData(&tamu[i]);
        ditemukan = true;
    }
}
} else {
    cout << "Format kamar salah!\n";
}
} else if (cari == 4) {
    printData(&tamu[jumlahTamu - 1]);
    ditemukan = true;
} else {
    cout << "Pilihan tidak valid!\n";
}

if (!ditemukan && cari >= 1 && cari <= 4) {
    cout << "Data tidak ditemukan.\n";
}

cout << "-----
\n\n";
system("pause");
}

```

3.3.5 Fitur Laporan Data Tamu

Fitur laporan digunakan untuk menampilkan data tamu berdasarkan kategori tertentu, yaitu tamu yang masih menginap, tamu yang sudah *check-out*, atau seluruh data tamu. Sebelum laporan ditampilkan, data akan diurutkan terlebih dahulu menggunakan algoritma *Bubble Sort* berdasarkan ID tamu agar informasi lebih rapi dan mudah dibaca.

```

void laporan(){
    header();
    cout << "\n----- Laporan Data Tamu -----
-----\n\n";

    if (jumlahTamu == 0) {
        cout << "Data tamu kosong.\n";
        system("pause");
        return;
    }

    bubbleSort();
    for (int i = 0; i < jumlahTamu; i++) {
        printData(&tamu[i]);
    }

    cout << "-----
\n";
    cout << "Total Tamu : " << jumlahTamu << " orang" << endl;
}

```

```

        cout << "-----\n\n";
        system("pause");
    }

    void bubbleSort() {
        for (int i = 0; i < jumlahTamu - 1; i++) {
            for (int j = 0; j < jumlahTamu - i - 1; j++) {
                if (tamu[j].id > tamu[j + 1].id) {
                    Tamu temp = tamu[j];
                    tamu[j] = tamu[j + 1];
                    tamu[j + 1] = temp;
                }
            }
        }
    }
}

```

3.2.6 Fitur Hitung Total Pendapatan

Fitur ini digunakan untuk menghitung total pemasukan hotel dari seluruh data tamu yang tersimpan. Sistem akan menghitung biaya menginap setiap tamu berdasarkan lama menginap, kemudian menjumlahkan seluruh biaya tersebut untuk memperoleh total pendapatan hotel. Selain itu, sistem juga menampilkan jumlah tamu dan rata-rata pendapatan per tamu.

```

int hitungBiaya(int hari) {
    int harga = 300000;
    if(hari <= 0)
        return 0;

    return harga + hitungBiaya(hari - 1);
}

void totalPendapatan() {
    header();
    cout << "\n----- Total Pendapatan -----
    -----\n\n";

    if (jumlahTamu == 0) {
        cout << "Data tamu kosong. Belum ada pendapatan\n";
        system("pause");
        return;
    }

    int total = 0;

    cout << "Rincian pendapatan per tamu:\n";
    cout << "-----
    -----\n";

    for(int i = 0; i < jumlahTamu; i++) {

```

```

        int biayaTamu = hitungBiaya(tamu[i].lamaInap);
        total += biayaTamu;

        cout << i + 1 << ". "
              << tamu[i].nama
              << " - " << tamu[i].lamaInap << " malam : Rp "
        << biayaTamu << "\n";
    }

    cout << "-----\n";
    cout << "Total Tamu      : " << jumlahTamu << " orang"
    << endl;
    cout << "Total Pendapatan : Rp " << total << endl;
    cout << "Rata-rata/Tamu   : Rp " << (total / jumlahTamu)
    << endl;
    cout << "-----\n\n";
    system("pause");
}

```

3.3. Screenshot Program

Berikut merupakan rangkaian *screenshot* dari antarmuka program *Hotel Management System* saat dijalankan, yang mencakup menu utama hingga seluruh fitur fungsional sistem:

```

File ditemukan. Memuat data...
Press any key to continue . . .

```

Gambar 3.3.1 Tampilan Memuat File Data

```

=====
HOTEL MANAGEMENT SYSTEM
=====
1. Lihat Status Kamar
2. Booking Kamar
3. Check Out Kamar
4. Cari Data Tamu
5. Laporan Data Tamu (Sorting)
6. Hitung Total Pendapatan
7. Keluar Aplikasi
=====
Pilih menu [1-7] : 

```

Gambar 3.3.2 Tampilan Menu Utama

```

=====
HOTEL MANAGEMENT SYSTEM
=====

----- Lihat Kamar -----

[Kamar 1A KOSONG][Kamar 1B TERISI][Kamar 1C TERISI][Kamar 1D KOSONG][Kamar 1E KOSONG]
[Kamar 2A KOSONG][Kamar 2B KOSONG][Kamar 2C KOSONG][Kamar 2D KOSONG][Kamar 2E KOSONG]
[Kamar 3A TERISI][Kamar 3B KOSONG][Kamar 3C KOSONG][Kamar 3D KOSONG][Kamar 3E TERISI]

Press any key to continue . . . █

```

Gambar 3.3.3 Tampilan Denah Status Kamar

```

=====
HOTEL MANAGEMENT SYSTEM
=====

----- Booking Kamar -----

Data tamu :
-----
ID : 12 (Otomatis)
Nama : Budi
Nomor Kamar (Contoh: 1A): 3A
Kamar 3A sudah terisi! Silakan pilih yang kosong.
Nomor Kamar (Contoh: 1A): 3C
Lama Inap : 3
Berhasil booking kamar
-----

Press any key to continue . . . █

```

Gambar 3.3.4 Fitur Booking Kamar

```

=====
HOTEL MANAGEMENT SYSTEM
=====

----- Check Out Kamar -----

Masukkan ID Tamu : 12

=====
INVOICE CHECK OUT
=====
ID          : 12
Nama        : Budi
Kamar       : 3C
Lama Inap   : 3 malam
Total Biaya : Rp 900000
=====

Check Out Berhasil! Kamar sudah dikosongkan.
-----

Press any key to continue . . . █

```

Gambar 3.3.5 Fitur Check Out Kamar

```

=====
HOTEL MANAGEMENT SYSTEM
=====

----- Cari Data Tamu -----

Pilih kategori pencarian:
1. ID
2. Nama
3. Kamar
4. Terakhir Booking
Pilih menu [1-4] : █

```

Gambar 3.3.6 Fitur Cari Data Tamu

```
=====
HOTEL MANAGEMENT SYSTEM
=====

----- Cari Data Tamu -----

Pilih kategori pencarian:
1. ID
2. Nama
3. Kamar
4. Terakhir Booking
Pilih menu [1-4] : 1

Hasil Pencarian:
Masukkan ID : 12
-----
ID          : 12
Nama        : Budi
Lantai      : 3
Kamar       : C
Lama Inap   : 3
Status      : Sudah Check Out
-----

Press any key to continue . . . █

Gambar 3.3.7 Hasil Pencarian Berdasarkan ID
```

```
=====
HOTEL MANAGEMENT SYSTEM
=====

----- Cari Data Tamu -----

Pilih kategori pencarian:
1. ID
2. Nama
3. Kamar
4. Terakhir Booking
Pilih menu [1-4] : 2

Hasil Pencarian:
Masukkan Nama : Budi
-----
ID          : 12
Nama        : Budi
Lantai      : 3
Kamar       : C
Lama Inap   : 3
Status      : Sudah Check Out
-----

Press any key to continue . . . █

Gambar 3.3.8 Hasil Pencarian Berdasarkan Nama
```



```
=====
HOTEL MANAGEMENT SYSTEM
=====

----- Cari Data Tamu -----

Pilih kategori pencarian:
1. ID
2. Nama
3. Kamar
4. Terakhir Booking
Pilih menu [1-4] : 3

Hasil Pencarian:
Masukkan Nomor Kamar (Contoh: 1A) : 3C
-----
ID          : 12
Nama        : Budi
Lantai      : 3
Kamar       : C
Lama Inap   : 3
Status      : Sudah Check Out
-----

Press any key to continue . . . █
```

Gambar 3.3.9 Hasil Pencarian Berdasarkan Nomor Kamar

```

=====
HOTEL MANAGEMENT SYSTEM
=====

----- Cari Data Tamu -----

Pilih kategori pencarian:
1. ID
2. Nama
3. Kamar
4. Terakhir Booking
Pilih menu [1-4] : 4

Hasil Pencarian:
-----
ID          : 12
Nama        : Budi
Lantai      : 3
Kamar       : C
Lama Inap   : 3
Status      : Sudah Check Out
-----

Press any key to continue . . . █

```

Gambar 3.3.10 Hasil Pencarian Tamu Terakhir Booking

```

=====
HOTEL MANAGEMENT SYSTEM
=====

----- Laporan Data Tamu -----

Pilih filter laporan:
1. Tamu Aktif
2. Tamu Check Out
3. Semua Tamu
Pilih filter [1-3] : █

```

Gambar 3.3.11 Laporan Data Tamu Terurut

```
=====
HOTEL MANAGEMENT SYSTEM
=====

----- Laporan Data Tamu -----

Pilih filter laporan:
1. Tamu Aktif
2. Tamu Check Out
3. Semua Tamu
Pilih filter [1-3] : 1

-----

ID      : 1
Nama    : Tessa Hasiantha
Lantai  : 1
Kamar   : C
Lama Inap : 3
Status  : Aktif/Menginap
-----

ID      : 9
Nama    : Yanto
Lantai  : 3
Kamar   : A
Lama Inap : 2
Status  : Aktif/Menginap
-----

ID      : 10
Nama    : fani
Lantai  : 3
Kamar   : E
Lama Inap : 6
Status  : Aktif/Menginap
-----

Total Tamu Ditampilkan : 3 orang
-----

Press any key to continue . . . █
```

Gambar 3.3.12 Tampilan Laporan Data Tamu Aktif



HOTEL MANAGEMENT SYSTEM	
=====	
----- Laporan Data Tamu -----	
Pilih filter laporan:	
1. Tamu Aktif	
2. Tamu Check Out	
3. Semua Tamu	
Pilih filter [1-3] : 3	

ID : 1	ID : 8
Nama : Tessa Hasiantha	Nama : eca fani
Lantai : 1	Lantai : 2
Kamar : C	Kamar : E
Lama Inap : 3	Lama Inap : 4
Status : Aktif/Menginap	Status : Sudah Check Out

ID : 2	ID : 9
Nama : Rifani Juniarti	Nama : Yanto
Lantai : 2	Lantai : 3
Kamar : A	Kamar : A
Lama Inap : 5	Lama Inap : 2
Status : Sudah Check Out	Status : Aktif/Menginap

ID : 10	ID : 12
Nama : fani	Nama : Budi
Lantai : 3	Lantai : 3
Kamar : E	Kamar : C
Lama Inap : 6	Lama Inap : 3
Status : Aktif/Menginap	Status : Sudah Check Out

Total Tamu Ditampilkan : 6 orang	
=====	

Gambar 3.3.14 Tampilan Laporan Semua Data Tamu

HOTEL MANAGEMENT SYSTEM	
=====	
----- Total Pendapatan -----	
Rincian pendapatan per tamu:	

1. Tessa Hasiantha - 3 malam : Rp 900000	
2. Rifani Juniarti - 5 malam : Rp 1500000	
3. eca fani - 4 malam : Rp 1200000	
4. Yanto - 2 malam : Rp 600000	
5. fani - 6 malam : Rp 1800000	
6. Budi - 3 malam : Rp 900000	

Total Tamu : 6 orang	
Total Pendapatan : Rp 6900000	
Rata-rata/Tamu : Rp 1150000	

Press any key to continue . . .	

Gambar 3.3.15 Tampilan Rincian Akumulasi Keuangan

```

=====
HOTEL MANAGEMENT SYSTEM
=====
1. Lihat Status Kamar
2. Booking Kamar
3. Check Out Kamar
4. Cari Data Tamu
5. Laporan Data Tamu (Sorting)
6. Hitung Total Pendapatan
7. Keluar Aplikasi
=====
Pilih menu [1-7] : 7
Apakah anda yakin ingin keluar? (y/n) : y
Terima kasih telah menggunakan program!

```

Gambar 3.3.16 Tampilan Keluar Sistem

3.4. Source Code Lengkap

```

#include <iostream>
#include <fstream>
#include <string>
#include <cctype>
using namespace std;

struct Tamu {
    int id;
    string nama;
    int lantai;
    int kamar;
    int lamaInap;
    bool status;
};

bool hotel[3][5] = {0};
const int maxTamu = 200;
Tamu tamu[maxTamu];
int jumlahTamu = 0;
int nextId = 1;

void header();
void lihatKamar();
void bookingKamar();
void checkOutKamar();
void cariTamu();
void laporan();
void loadFile();
bool saveFile(Tamu* t);
void rewriteFile();
void totalPendapatan();

```

```

int hitungBiaya(int hari);
void printData(Tamu* t);
void bubbleSort();
string spasiToUnderscore(string teks);
string underscoreToSpasi(string teks);
string stringToupper(string text);
char getAbjadKamar(int index);
int getIndexKamar(char huruf);

int main() {
    system("cls");
    loadFile();

    int pilih;
    char out;
    bool keluar = false;

    do {
        header();

        cout << "1. Lihat Status Kamar\n"
              << "2. Booking Kamar\n"
              << "3. Check Out Kamar\n"
              << "4. Cari Data Tamu\n"
              << "5. Laporan Data Tamu (Sorting)\n"
              << "6. Hitung Total Pendapatan\n"
              << "7. Keluar Aplikasi\n"
              << "=====\\n"
              << "Pilih menu [1-7] : ";

        cin >> pilih;

        switch (pilih) {
            case 1 :
                lihatKamar();
                break;
            case 2 :
                bookingKamar();
                break;
            case 3 :
                checkOutKamar();
                break;
            case 4 :
                cariTamu();
                break;
            case 5 :
                laporan();
                break;
            case 6 :
                totalPendapatan();
                break;
            case 7 :
                cout << "Apakah anda yakin ingin keluar? (y/n) : ";

```

```

        cin >> out;
        if (out == 'y' || out == 'Y') {
            cout << "Terima kasih telah menggunakan
program!\n";
            keluar = true;
        }
        break;
    default :
        cout << "Menu tidak valid!\n";
        system("pause");
        break;
    }
} while (keluar == false);
return 0;
}

void header(){
    system("cls");
    cout << "=====\n"
        << "                HOTEL MANAGEMENT SYSTEM                \n"
        << "=====\n";
}

string spasiToUnderscore(string teks) {
    for(int i = 0; i < teks.length(); i++) {
        if(teks[i] == ' ') {
            teks[i] = '_';
        }
    }
    return teks;
}

string underscoreToSpasi(string teks) {
    for(int i = 0; i < teks.length(); i++) {
        if(teks[i] == '_') {
            teks[i] = ' ';
        }
    }
    return teks;
}

string stringToupper(string text){
    for(char& word : text){
        word = (char)toupper(word);
    }
    return text;
}

char getAbjadKamar(int index) {
    string abjad = "ABCDE";
    return abjad[index];
}

```



```

int getIndexKamar(char huruf) {
    string abjad = "ABCDE";
    for (int i = 0; i < 5; i++) {
        if (abjad[i] == toupper(huruf)) {
            return i;
        }
    }
    return -1;
}

void printData(Tamu* t) {
    char noKamar = getAbjadKamar(t->kamar);
    string status = (t->status) ? "Aktif/Menginap" : "Sudah Check Out";

    cout << "-----\n"
         << "ID          : " << t->id << "\n"
         << "Nama          : " << t->nama << "\n"
         << "Lantai        : " << t->lantai + 1 << "\n"
         << "Kamar          : " << noKamar << "\n"
         << "Lama Inap      : " << t->lamaInap << "\n"
         << "Status        : " << status << "\n";
}

void lihatKamar(){
    header();
    cout << "\n----- Lihat Kamar -----
\n\n";

    string statusKamar;
    for (int i = 0; i < 3; i++){
        for (int j = 0; j < 5; j++){
            statusKamar = hotel[i][j] ? "TERISI" : "KOSONG";
            cout << "[Kamar " << i+1 << getAbjadKamar(j) << " " <<
statusKamar << "]\n";
        }
        cout << endl;
    }

    cout << endl;
    system("pause");
}

/**
 * Input Data
 */
void bookingKamar(){
    header();
    cout << "\n----- Booking Kamar -----
\n\n";

    if (jumlahTamu >= maxTamu) {
        cout << "Maaf, kapasitas tamu penuh (maksimal " << maxTamu << "
tamu)!\n";
    }
}

```

```

        system("pause");
        return;
    }

    Tamu inputTamu;
    string noKamar;

    cout << "Data tamu :\n";
    cout << "-----\n";

    inputTamu.id = nextId;
    cout << "ID : " << inputTamu.id << " (Otomatis)\n";

    cout << "Nama : ";
    cin.ignore();
    getline(cin, inputTamu.nama);

    bool kamarValid = false;
    do {
        cout << "Nomor Kamar (Contoh: 1A): ";
        cin >> noKamar;

        if (noKamar.length() != 2) {
            cout << "Format salah harus 2 karakter (Contoh: 1A).\n";
            continue;
        }

        inputTamu.lantai = noKamar[0] - '1';
        inputTamu.kamar = getIndexKamar(noKamar[1]);

        if (inputTamu.lantai < 0 || inputTamu.lantai > 2 ||
            inputTamu.kamar == -1 || toupper(noKamar[1]) > 'E') {
            cout << "Kamar tidak ada! Pilih lantai 1-3 dan abjad A-
E.\n";
            continue;
        }

        if (hotel[inputTamu.lantai][inputTamu.kamar] == true) {
            cout << "Kamar " << noKamar << " sudah terisi! Silakan
pilih yang kosong.\n";
            continue;
        }
        kamarValid = true;
    }while (kamarValid == false);

    cout << "Lama Inap : "; cin >> inputTamu.lamaInap;
    inputTamu.status = true;

    if (saveFile(&inputTamu) == true) {
        tamu[jumlahTamu] = inputTamu;
        hotel[inputTamu.lantai][inputTamu.kamar] = 1;
        jumlahTamu++;
    }

```

```

        nextId++;
        cout << "Berhasil booking kamar\n";
    } else {
        cout << "Gagal booking kamar\n";
    }

    cout << "-----\n\n";
    system("pause");
}

/**
 * Update Data
 */
void checkOutKamar() {
    header();
    cout << "\n----- Check Out Kamar -----
\n\n";

    if (jumlahTamu == 0) {
        cout << "Data tamu kosong. Tidak ada yang bisa dicheckout.\n";
        system("pause");
        return;
    }

    int id;
    cout << "Masukkan ID Tamu : ";
    cin >> id;

    bool ditemukan = false;
    for (int i = 0; i < jumlahTamu; i++) {
        if (tamu[i].id == id) {
            ditemukan = true;
            if (!tamu[i].status) {
                cout << "Tamu dengan ID " << id << " sudah check
out.\n";
            } else {
                int tagihan = hitungBiaya(tamu[i].lamaInap);

                cout <<
"\n=====
\n";
                cout << "
                                INVOICE CHECK OUT
\n";
                cout <<
"=====
\n";
                cout << "ID          : " << tamu[i].id << "\n";
                cout << "Nama          : " << tamu[i].nama << "\n";
                cout << "Kamar         : " << tamu[i].lantai + 1 <<
getAbjadKamar(tamu[i].kamar) << "\n";
                cout << "Lama Inap     : " << tamu[i].lamaInap << "
malam\n";
                cout << "Total Biaya : Rp " << tagihan << "\n";
                cout <<
"=====
\n";

```

```

        tamu[i].status = false;
        hotel[tamu[i].lantai][tamu[i].kamar] = 0;
        rewriteFile();

        cout << "\nCheck Out Berhasil! Kamar sudah
dikosongkan.\n";
    }
    break;
}

}

if (!ditemukan) {
    cout << "Tamudengan ID " << id << " tidak ditemukan.\n";
}

cout << "-----\n\n";
system("pause");
}

/**
 * Searching Data
 */
void cariTamu(){
    header();
    cout << "\n----- Cari Data Tamu -----
\n\n";

    if (jumlahTamu == 0) {
        cout << "Data tamu kosong. Tidak ada data yang bisa dicari.
Silakan booking kamar\n";
        system("pause");
        return;
    }

    int cari;
    cout << "Pilih kategori pencarian:\n"
        << "1. ID\n"
        << "2. Nama\n"
        << "3. Kamar\n"
        << "4. Terakhir Booking\n"
        << "Pilih menu [1-4] : ";
    cin >> cari;

    bool ditemukan = false;
    cout << "\nHasil Pencarian:\n";

    if (cari == 1) { // binary search
        int keyword;
        cout << "Masukkan ID : ";
        cin >> keyword;

        bubbleSort();

```

```

    int awal = 0;
    int akhir = jumlahTamu - 1;
    int tengah;

    while (awal <= akhir) {
        tengah = (awal + akhir) / 2;

        if (tamu[tengah].id == keyword) {
            printData(&tamu[tengah]);
            ditemukan = true;
            break;
        } else if (tamu[tengah].id < keyword) {
            awal = tengah + 1;
        } else {
            akhir = tengah - 1;
        }
    }
} else if (cari == 2) { // linear search
    string keyword;
    cout << "Masukkan Nama : ";
    cin.ignore();
    getline(cin, keyword);
    string upperKeyword = stringToupper(keyword);
    for (int i = 0; i < jumlahTamu; i++) {
        if (stringToupper(tamu[i].nama) == upperKeyword) {
            printData(&tamu[i]);
            ditemukan = true;
        }
    }
} else if (cari == 3) { // linear search
    string keyword;
    cout << "Masukkan Nomor Kamar (Contoh: 1A) : ";
    cin >> keyword;
    if (keyword.length() == 2) {
        int lantaiCari = keyword[0] - '1';
        int kamarCari = getIndexKamar(keyword[1]);
        for (int i = 0; i < jumlahTamu; i++) {
            if (tamu[i].lantai == lantaiCari && tamu[i].kamar ==
kamarCari) {
                printData(&tamu[i]);
                ditemukan = true;
            }
        }
    } else {
        cout << "Format kamar salah!\n";
    }
} else if (cari == 4) {
    printData(&tamu[jumlahTamu - 1]);
    ditemukan = true;
} else {
    cout << "Pilihan tidak valid!\n";
}

```

```

    if (!ditemukan && cari >= 1 && cari <= 4) {
        cout << "Data tidak ditemukan.\n";
    }

    cout << "-----\n\n";
    system("pause");
}

/**
 * Sorting Data
 */
void laporan(){
    header();
    cout << "\n----- Laporan Data Tamu -----
\n\n";

    if (jumlahTamu == 0) {
        cout << "Data tamu kosong.\n";
        system("pause");
        return;
    }

    int filter;
    cout << "Pilih filter laporan:\n"
        << "1. Tamu Aktif\n"
        << "2. Tamu Check Out\n"
        << "3. Semua Tamu\n"
        << "Pilih filter [1-3] : ";
    cin >> filter;

    bubbleSort();

    int count = 0;
    cout << "\n";

    for (int i = 0; i < jumlahTamu; i++) {
        if (filter == 1 && tamu[i].status == true) {
            printData(&tamu[i]);
            count++;
        } else if (filter == 2 && tamu[i].status == false) {
            printData(&tamu[i]);
            count++;
        } else if (filter == 3) {
            printData(&tamu[i]);
            count++;
        }
    }

    if (filter < 1 || filter > 3) {
        cout << "Pilihan tidak valid!\n";
    }
}

```

```

        cout << "-----\n";
        cout << "Total Tamu Ditampilkan : " << count << " orang" << endl;
        cout << "-----\n\n";
        system("pause");
    }

    void bubbleSort() {
        for (int i = 0; i < jumlahTamu - 1; i++) {
            for (int j = 0; j < jumlahTamu - i - 1; j++) {
                if (tamu[j].id > tamu[j + 1].id) {
                    Tamu temp = tamu[j];
                    tamu[j] = tamu[j + 1];
                    tamu[j + 1] = temp;
                }
            }
        }
    }

    /**
     * Rekursif hitung pendapatan
     */
    int hitungBiaya(int hari) {
        int harga = 300000;
        if(hari <= 0)
            return 0;

        return harga + hitungBiaya(hari - 1);
    }

    void totalPendapatan() {
        header();
        cout << "\n----- Total Pendapatan -----
\n\n";

        if (jumlahTamu == 0) {
            cout << "Data tamu kosong. Belum ada pendapatan\n";
            system("pause");
            return;
        }

        int total = 0;

        cout << "Rincian pendapatan per tamu:\n";
        cout << "-----\n";

        for(int i = 0; i < jumlahTamu; i++) {
            int biayaTamu = hitungBiaya(tamu[i].lamaInap);
            total += biayaTamu;

            cout << i + 1 << ". "
                << tamu[i].nama
                << " - " << tamu[i].lamaInap << " malam : Rp " << biayaTamu
                << "\n";

```

```

    }

    cout << "-----\n";
    cout << "Total Tamu      : " << jumlahTamU << " orang" << endl;
    cout << "Total Pendapatan : Rp " << total << endl;
    cout << "Rata-rata/Tamu    : Rp " << (total / jumlahTamU) << endl;
    cout << "-----\n\n";
    system("pause");
}

/**
 * File
 */
void loadFile() {
    ifstream file("data_tamu.txt");
    if (file.is_open()){
        cout << "File ditemukan. Memuat data...\n";
        int maxId = 0;
        while (jumlahTamU < maxTamU && file >> tamu[jumlahTamU].id >>
tamu[jumlahTamU].nama >> tamu[jumlahTamU].lantai >>
tamu[jumlahTamU].kamar >> tamu[jumlahTamU].lamaInap >>
tamu[jumlahTamU].status) {

            tamu[jumlahTamU].nama =
underscoreToSpasi(tamu[jumlahTamU].nama);

            if (tamu[jumlahTamU].status) {
                hotel[tamu[jumlahTamU].lantai][tamu[jumlahTamU].kamar]
= 1;
            }

            if (tamu[jumlahTamU].id > maxId) {
                maxId = tamu[jumlahTamU].id;
            }

            jumlahTamU++;
        }
        nextId = maxId + 1;
        file.close();
    } else {
        cout << "File tidak ditemukan! Memulai program dengan data
kosong\n";
    }
    system("pause");
}

bool saveFile(Tamu* t) {
    ofstream file("data_tamu.txt", ios::app);

    if(!file.is_open()){
        return false;
    } else {
        file << t->id << " "

```



```

        << spasiToUnderscore(t->nama) << " "
        << t->lantai << " "
        << t->kamar << " "
        << t->lamaInap << " "
        << t->status
        << endl;
    file.close();
    return true;
}
}

void rewriteFile() {
    ofstream file("data_tamu.txt", ios::trunc);

    if(file.is_open()){
        for (int i = 0; i < jumlahTamu; i++) {
            file << tamu[i].id << " "
                << spasiToUnderscore(tamu[i].nama) << " "
                << tamu[i].lantai << " "
                << tamu[i].kamar << " "
                << tamu[i].lamaInap << " "
                << tamu[i].status
                << endl;
        }
        file.close();
    }
}
}

```

BAB IV

JADWAL Pengerjaan Tugas dan Pembagian Tugas

4.1. Jadwal Pengerjaan

Tabel 4.1 Tabel Jadwal Pengerjaan

No.	Kegiatan	Mei			Juni
		2	3	4	1
1.	Mengkaji Masalah dan Analisis Kebutuhan Program				
2.	Perancangan Sistem Program, Alur, dan Struktur Data				
3.	Pengodean Program				
4.	Pengujian dan <i>Debugging</i>				
5.	Revisi Program				
6.	Penyusunan Laporan				

4.2. Pembagian Tugas

Kami membagi tugas sesuai dengan tanggung jawab masing masing setiap kegiatan.

Tabel 4.2 Tabel Pembagian Tugas

No.	Kegiatan	Penanggung Jawab
1.	Mengkaji Masalah dan Analisis Kebutuhan Program	Rifani Juniarti dan Tessa Hasiantha
2.	Perancangan Sistem Program, Alur, dan Struktur Data	Rifani Juniarti dan Tessa Hasiantha
3.	Pengembangan Menu Lihat Kamar, Check Out Kamar, Laporan, dan Keluar	Tessa Hasiantha
4.	Pengembangan Menu Booking Kamar, Cari Data, Hitung Total Pendapatan, dan File I/O	Rifani Juniarti
5.	Pengujian dan <i>Debugging</i>	Rifani Juniarti dan Tessa Hasiantha
6.	Revisi Program	Rifani Juniarti

7.	Penyusunan Laporan	Rifani Juniarti dan Tessa Hasiantha
----	--------------------	--

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan terhadap aplikasi *Hotel Management System* berbasis C++, maka dapat diambil beberapa kesimpulan sebagai berikut:

1. Aplikasi berhasil dibangun dengan menerapkan struktur data berupa *struct*, *array* satu dimensi untuk data reservasi, serta *array* dua dimensi untuk memetakan status keterisian 15 kamar hotel.
2. Seluruh fitur utama dapat berfungsi dengan baik dan saling terintegrasi.
3. Penerapan algoritma *Binary Search* dan *Linear Search* terbukti efektif mempercepat pencarian data, didukung oleh *Bubble Sort* yang menyusun laporan data tamu secara rapi berdasarkan.
4. Penggunaan konsep rekursi mempermudah perhitungan biaya menginap, sementara *File Handling (ifstream/ofstream)* berhasil menjaga data tetap tersimpan permanen dalam file `data_tamu.txt`.
5. Sistem ini mampu menjadi solusi digital untuk mengurangi pencatatan manual dan meningkatkan efisiensi operasional hotel.

5.2 Saran

Beberapa saran yang dapat diberikan untuk pengembangan sistem ini di masa mendatang adalah:

1. Mengembangkan antarmuka program dari berbasis teks (CLI) menjadi berbasis grafis (GUI) agar visualisasi denah kamar dan invoice lebih interaktif.
2. Memperkuat fungsi error handling pada validasi input untuk mencegah program crash akibat kesalahan pengetikan oleh pengguna.
3. Menambahkan fitur variasi tipe kamar (seperti *Standard*, *Deluxe*, dll.) dengan tarif sewa per malam yang berbeda.
4. Menambahkan sistem login akun dan enkripsi file data guna meningkatkan keamanan dan kerahasiaan informasi tamu.

DAFTAR PUSTAKA

Link Repositori:

<https://github.com/rifanijuni/hotel-management-system>